08 Web Application Firewall Middleware

08-waf-middleware

Bounded Flask WAF with four rule families, scored anomalies, redacted logging and reproducible local benchmark

Verified here: 42 passed; 0 failed; 0 skipped

Measured results

- 296/386 (76.7%) held-out public payload strings detected
- 0/35 hand-written benign strings flagged
- 42 passing pytest cases; Streamlit AppTest has no exceptions
- SQLi 57/100; XSS 78/86; traversal 63/100; command 98/100
- Median local test-client latency 0.234 ms

Data provenance and caveats

- SecLists (MIT), seeded curated subset: https://github.com/danielmiessler/SecLists; exact raw URLs in samples/payloads/*.txt
- 35 hand-written benign strings in samples/benign.txt
- Public payload-list labels include inert and context-specific strings; no exploit success was measured.
- The 35-example benign set is hand-written, not production traffic; 0/35 is not a general false-positive estimate.
- All requests go only to the project's own Flask test client; no external target.
- Heuristic pattern matching is not a production WAF or bypass guarantee.

Project contents

README.md, requirements.txt, cli.py, app.py, src/__init__.py, src/benchmark.py, src/engine.py, src/middleware.py, tests/test_benchmark.py, tests/test_engine.py, tests/test_middleware.py, .gitignore, pytest.ini, scripts/fetch_payloads.py, scripts/render_chart.py

Sentinel WAF Middleware

A bounded Flask request-inspection layer with anomaly scoring, block/monitor modes, redacted JSONL audit events, per-rule counts, and a local, reproducible benchmark.

Problem

Even small web services need a way to observe suspicious input before it reaches routes. This educational middleware scores SQL injection, XSS, directory traversal and command-injection patterns across path, query, text/JSON/form bodies and non-sensitive header values. It is **not a production-grade WAF**: pattern rules do not understand application context, do not prove exploitation and give no bypass guarantee.

Architecture

```text
HTTP request ? Flask before_request ? size/field guards ? NFKC + ?2 URL-decode rounds
                                                  ?
                                       bounded regex rules ? anomaly score
                                                  ?
                                redacted JSONL audit + per-rule hits ? allow / 403

CLI benchmark ? own Flask test client only ? JSON + HTML + PNG ? Streamlit viewer
```

`src/engine.py` contains compiled rules with IDs, families, severities and points. Each rule scores at most once per field even if it matches multiple canonical forms. At default threshold 6, high/critical rules block on their own; weaker evidence can add up.
`src/middleware.py` extracts fields and installs a `before_request` hook on any Flask app. Monitor mode scores and logs but passes requests on. Thresholds 1?100 are configurable.

Security design decisions

- Reject inputs exceeding 16,384 total field bytes, 4,096 bytes per field or 64 fields in block mode; monitor mode records a limit reason but passes. Body reads are capped at 16 KiB. JSON nesting is bounded to depth six. Binary/multipart/unknown content types and malformed JSON explicitly fail closed in block mode, rather than claiming coverage.
- Decode with Unicode NFKC, then at most two URL-decoding rounds per field. Flask has already decoded parsed query/form values once; `RAW_URI` is used for the path when available. How reverse proxies normalize requests may differ. HTML entity encoding and arbitrary multilayer encodings are not exhaustively handled.
- Regex input is bounded; patterns avoid nested unbounded repetition and constrain wildcard spans. A manual worst-case check on five 4 KiB candidate strings ran at ~0.7?2.1 ms each on the build machine, not a formal proof against every adversarial string.
- Audit JSONL events contain only mode, score, block outcome, rule IDs, field types and fixed limit reasons. Raw body, query, header values, path, user-provided keys, IPs and credentials are not logged. Authorization/Cookie/Proxy-Authorization/Set-Cookie values are skipped during scanning entirely. Route code must separately sanitize output and parameterize SQL.
- Rules and payload benchmark are defensive local demonstrations only. No attack strings are sent to an external target.

Project structure

```text
08-waf-middleware/
??? app.py                # read-only Streamlit results dashboard
??? cli.py                # benchmark and demo commands
??? src/{engine,middleware,benchmark}.py
??? scripts/{fetch_payloads,render_chart}.py
```

```
??? samples/benign.txt     # hand-written safe examples
??? samples/payloads/      # seeded subset from public SecLists, with source URLs
??? tests/                 # local pytest suite
??? outputs/               # real benchmark reports, chart and test log
```

Setup and run

```bash
python -m venv .venv
. .venv/bin/activate       # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python cli.py --help
python cli.py demo
python cli.py demo --mode monitor --threshold 8
python cli.py benchmark --split holdout --mode block --threshold 6
python scripts/render_chart.py
pytest -v
streamlit run app.py
```

The benchmark uses Flask's local test client; no server or network is needed at run time.
To refresh public sample files (does access GitHub), run `python scripts/fetch_payloads.py
 --seed 7 --per-family 200`. SecLists repository:
https://github.com/danielmiessler/SecLists ; source URLs appear in each
`samples/payloads/*.txt` header. Selected files:

- https://raw.githubusercontent.com/danielmiessler/SecLists/master/Fuzzing/Databases/SQLi/Generic-SQLi.txt
- https://raw.githubusercontent.com/danielmiessler/SecLists/master/Fuzzing/Databases/SQLi/quick-SQLi.txt
- https://raw.githubusercontent.com/danielmiessler/SecLists/master/Fuzzing/Databases/SQLi/Generic-BlindSQLi.fuzzdb.txt
- https://raw.githubusercontent.com/danielmiessler/SecLists/master/Fuzzing/XSS/human-friendly/XSS-Jhaddix.txt
- https://raw.githubusercontent.com/danielmiessler/SecLists/master/Fuzzing/XSS/human-friendly/XSS-RSNAKE.txt
- https://raw.githubusercontent.com/danielmiessler/SecLists/master/Fuzzing/LFI/LFI-Jhaddix.txt
- https://raw.githubusercontent.com/danielmiessler/SecLists/master/Fuzzing/command-injection-commix.txt
- https://raw.githubusercontent.com/danielmiessler/SecLists/master/Fuzzing/UnixAttacks.fuzzdb.txt

Subset is selected by deterministic seed 7, split into dev/holdout within each family, and
 each sample is submitted once as a JSON string to our own `/api/echo` route. This route
never executes or reflects it. The first 100 of each 200 selected SQLi/traversal/command
payloads and first 86 of 172 XSS are dev; the rest are holdout. `samples/benign.txt` holds
 35 hand-written safe strings. The source list's labels include invalid and context-
dependent strings. The benchmark measures rule matches for *these input lists*, not
exploit success or real-world detection.

To integrate into an existing Flask app:

```python
from pathlib import Path
from flask import Flask
from src.engine import Engine
from src.middleware import install_waf
```

```
app = Flask(__name__)
engine = Engine(mode="monitor", threshold=6)  # review logs before block mode
install_waf(app, engine, audit_path=Path("audit.jsonl"))
# app routes go here; your routes must still validate and encode their output.
```

Results, measured locally

- **42 pytest cases passed; 0 failed; 0 skipped.** The suite exercises all four rule families, encoding, NFKC, limits, block/monitor modes, redacted logging, query/path/body/header extraction and benchmark reports.
- **Holdout public-payload detection:** **296 / 386 (76.7%)** at threshold 6 in block mode. SQLi 57/100; XSS 78/86; traversal 63/100; command injection 98/100. These are local payload-list labels, not verified working exploits.
- **Hand-written benign false positives:** **0 / 35 (0.0%)**. This tiny, hand-picked set cannot estimate a production false-positive rate; confidence bounds are wide.
- Flask local test-client median **0.234 ms** across 421 requests on this box; excludes network/proxy overhead. Full per-rule hits and denominators in `outputs/benchmark.json`, self-contained `outputs/benchmark.html`, and `outputs/detection_by_family.png`.
- A separate dev split scored 289/386 (74.9%) with 0/35 benign false positives; development choices were evaluated on dev before reporting held-out results. No public data were used to train a model.

Sample output (real CLI run)

```text
holdout detection: 296/386 (76.7%)
benign false positives: 0/35 (0.0%)
  sqli: 57/100
  xss: 78/86
  traversal: 63/100
  cmdi: 98/100
Median latency: 0.234 ms; reports: outputs/benchmark.json, outputs/benchmark.html
```

Limitations & future work

Rules under-detect SQLi and traversal variants; base64-encoded or novel payloads may be missed. A low threshold increases false positives on natural language and markup. Some source strings are inert; the evaluation does not simulate backend parser behavior or determine whether an attack succeeds. The SQLi and traversal lists can contain context-specific strings (e.g., a bare filename). The chosen benign set is not representative of production traffic. Multipart uploads are not inspected; in block mode they are rejected. Header parsing and decoding may disagree with the reverse proxy. Monitoring sensitive operations needs careful deployment privacy reviews. This tool does not replace parameterized SQL, proper output encoding, access controls, a maintained WAF engine or professional security testing.

Next steps: real annotated, consented traffic; rule regression corpus and calibration; proxy normalization parity; context-aware HTML/SQL parsing; stable metrics storage (currently per-process); multipart handling with safe streaming.

Resume bullets

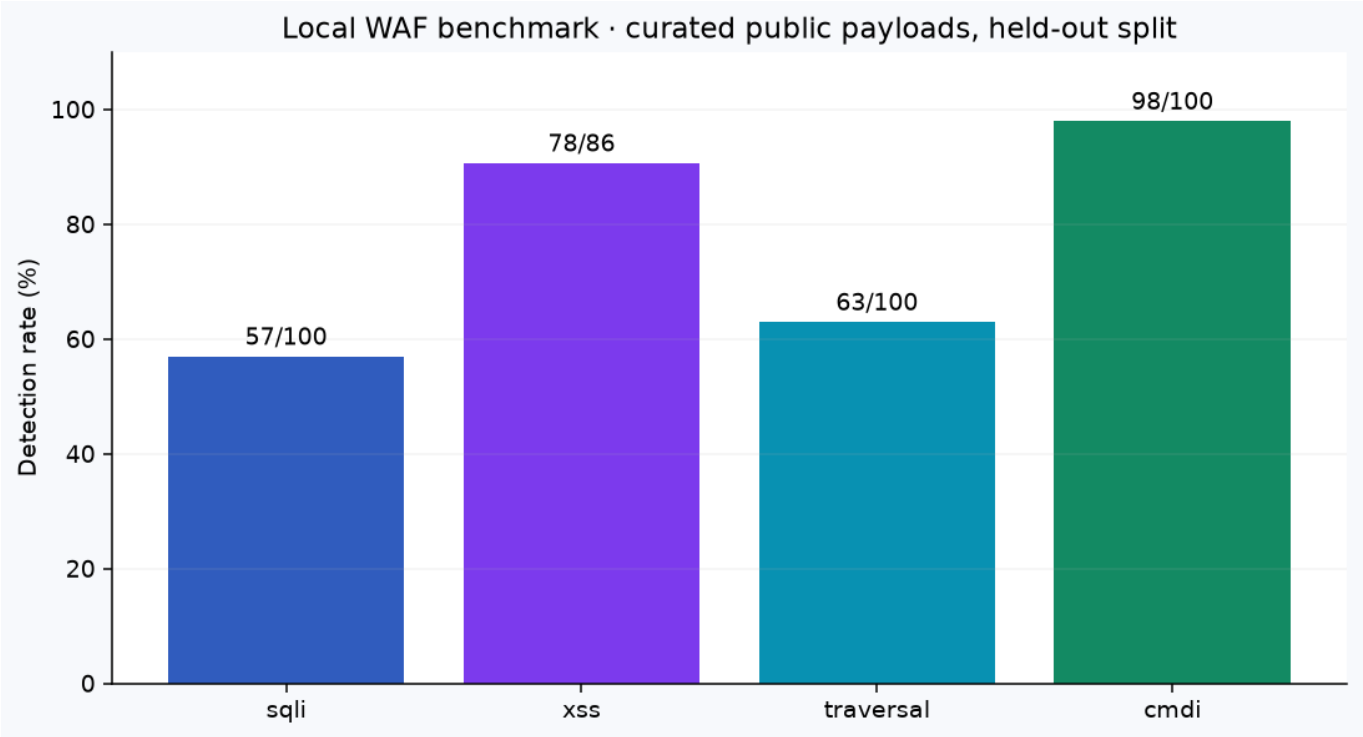
- Implemented a Flask WAF middleware with 12 scored rules across four attack families, configurable monitor/block modes, bounded canonicalization and privacy-safe JSONL audit events.
- Benchmarked 386 held-out curated SecLists payloads locally, flagging 296 (76.7%) at threshold 6; reported 0/35 false positives on a small hand-written benign set, with explicit dataset limitations.

- Wrote 42 passing tests covering encoded attacks, Unicode normalization, four request surfaces, size limits, monitor/block behavior and audit-log redaction.
- Built reproducible test-client benchmarking with per-rule counters, JSON/HTML reports, a PNG chart and a Streamlit metrics dashboard.

Web Application Firewall Middleware — ACTUAL SAMPLE OUTPUT

```
benign     HTTP 200 {'ok': True, 'waf_score': 0}
SQLi       HTTP 403 {'error': 'request rejected', 'score': 24}
XSS        HTTP 403 {'error': 'request rejected', 'score': 12}
traversal  HTTP 403 {'error': 'request rejected', 'score': 22}
command    HTTP 403 {'error': 'request rejected', 'score': 20}
rule hits: {'CMD02': 1, 'PATH01': 1, 'PATH02': 2, 'PATH03': 2, 'SQL01': 1, 'SQL02': 1, 'SQL03': 1,
 'SQL04': 1, 'XSS01': 1, 'XSS03': 1}

```


Local WAF benchmark · curated public payloads, held-out split
Detection rate (%)
0
20
40
60
80
100
57/100
78/86
63/100
98/100
sqli
xss
traversal
cmdi

08-waf-middleware/requirements.txt

```
Flask>=3.0,<4
requests>=2.31,<3
pytest>=8.0,<10
streamlit>=1.40,<2
plotly>=5.20,<7

```

```python
"""Local benchmark and WAF demonstration CLI; never attacks remote hosts."""
import argparse
from pathlib import Path
from src.benchmark import evaluate, save
from src.middleware import create_app


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    sub = parser.add_subparsers(dest="command", required=True)
    bench = sub.add_parser("benchmark", help="evaluate local test-client corpus")
    bench.add_argument("--split", choices=("dev", "holdout"), default="holdout")
    bench.add_argument("--mode", choices=("block", "monitor"), default="block")
    bench.add_argument("--threshold", type=int, default=6)
    bench.add_argument("--output", type=Path, default=Path("outputs"))
    demo = sub.add_parser("demo", help="send fixed synthetic requests locally")
    demo.add_argument("--mode", choices=("block", "monitor"), default="block")
    demo.add_argument("--threshold", type=int, default=6)
    args = parser.parse_args()
    if args.command == "benchmark":
        result = evaluate(args.split, args.threshold, args.mode)
        save(result, args.output)
        print(f"{result['split']} detection: {result['malicious']['detected']}/{result['malicious']['total']} ({result['malicious']['detection_rate']:.1%})")
        print(f"benign false positives: {result['benign']['false_positives']}/{result['benign']['total']} ({result['benign']['false_positive_rate']:.1%})")
        for family, values in result["families"].items():
            print(f"  {family}: {values['detected']}/{values['total']}")
        print(f"Median latency: {result['latency_ms_median']} ms; reports: {args.output / 'benchmark.json'}, {args.output / 'benchmark.html'}")
    else:
        app = create_app(mode=args.mode, threshold=args.threshold)
        client = app.test_client()
        for label, value in (("benign", "coffee and cake"), ("SQLi", "' OR 1=1--"),
                             ("XSS", "<script>alert(1)</script>"), ("traversal", "../../etc/passwd"),
                             ("command", "; cat /etc/passwd")):
            response = client.post("/api/echo", json={"input": value})
            print(f"{label:10s} HTTP {response.status_code} {response.get_json()}")
        print(f"rule hits: {app.extensions['waf_engine'].stats()['per_rule_hits']}")


if __name__ == "__main__":
    main()

```

```python
"""Read-only dashboard for local benchmark artifacts. No network needed."""
import json
from pathlib import Path
import plotly.express as px
import pandas as pd
import streamlit as st

st.set_page_config(page_title="WAF Metrics", layout="wide")
st.title("Web Application Firewall · local benchmark")
report_path = Path(__file__).parent / "outputs" / "benchmark.json"
if not report_path.exists():
    st.info("Run `python cli.py benchmark` to generate results.")
    st.stop()
r = json.loads(report_path.read_text(encoding="utf-8"))
a, b, c = st.columns(3)
a.metric("Malicious detection", f"{r['malicious']['detection_rate']:.1%}", f"{r['malicious']['detected']}/{r['malicious']['total']}")
b.metric("Benign false positives", f"{r['benign']['false_positive_rate']:.1%}", f"{r['benign']['false_positives']}/{r['benign']['total']}")
c.metric("Median request time (test client)", f"{r['latency_ms_median']} ms")
tab1, tab2 = st.tabs(["Detection", "Rules & limitations"])
with tab1:
    df = pd.DataFrame([{"family": k, **v} for k, v in r["families"].items()])
    fig = px.bar(df, x="family", y="rate", text="total", range_y=[0, 1], title="Detection by public-payload source family")
    fig.update_traces(texttemplate="n=%{text}")
    st.plotly_chart(fig, width="stretch")
    st.dataframe(df, hide_index=True)
with tab2:
    hits = pd.DataFrame([{"rule_id": k, "hits": v} for k, v in r["per_rule_hits"].items()])
    if not hits.empty:
        st.bar_chart(hits.set_index("rule_id"))
    st.caption("No raw paths, query values, headers, or body text are logged by the middleware.")
    for note in r["notes"]:
        st.write("•", note)

```

08-waf-middleware/src/__init__.py

```
1
```

```python
"""Local test-client benchmark; payloads never sent to public endpoints."""
from __future__ import annotations
from datetime import datetime, timezone
from html import escape
import json
from pathlib import Path
import statistics
import time
from .middleware import create_app

ROOT = Path(__file__).resolve().parent.parent
FAMILIES = ("sqli", "xss", "traversal", "cmdi")


def read_lines(path: Path) -> list[str]:
    return [line.strip() for line in path.read_text(encoding="utf-8").splitlines()
            if line.strip() and not line.startswith("#")]


def evaluate(split: str = "holdout", threshold: int = 6, mode: str = "block") -> dict:
    if split not in {"dev", "holdout"}:
        raise ValueError("split must be dev or holdout")
    app = create_app(mode=mode, threshold=threshold)
    client = app.test_client()
    by_family: dict[str, dict] = {}
    latencies: list[float] = []
    for family in FAMILIES:
        samples = read_lines(ROOT / "samples" / "payloads" / f"{family}_{split}.txt")
        detected = 0
        for sample in samples:
            start = time.perf_counter()
            response = client.post("/api/echo", json={"input": sample})
            latencies.append((time.perf_counter() - start) * 1000)
            detected += response.status_code == 403 if mode == "block" else response.get_json().ge
t("waf_score", 0) >= threshold
        by_family[family] = {"detected": detected, "total": len(samples),
                             "rate": round(detected / len(samples), 4) if samples else 0}
    benign = read_lines(ROOT / "samples" / "benign.txt")
    false_positives = 0
    for sample in benign:
        start = time.perf_counter()
        response = client.post("/api/echo", json={"input": sample})
        latencies.append((time.perf_counter() - start) * 1000)
        false_positives += response.status_code == 403 if mode == "block" else response.get_json()
.get("waf_score", 0) >= threshold
    malicious_total = sum(v["total"] for v in by_family.values())
    detected_total = sum(v["detected"] for v in by_family.values())
    engine = app.extensions["waf_engine"]
    return {
        "split": split, "mode": mode, "threshold": threshold,
        "dataset": "SecLists curated seeded subset vs hand-written benign examples",
        "malicious": {"detected": detected_total, "total": malicious_total,
                      "detection_rate": round(detected_total / malicious_total, 4)},
        "benign": {"false_positives": false_positives, "total": len(benign),
                   "false_positive_rate": round(false_positives / len(benign), 4)},
        "families": by_family,
        "latency_ms_median": round(statistics.median(latencies), 3),
        "latency_ms_max": round(max(latencies), 3),
        "per_rule_hits": engine.stats()["per_rule_hits"],
        "notes": ["Local Flask test-client benchmark only; source payload labels not independently
 verified.",
                  "Seeded sampling from SecLists; this is not a production efficacy guarantee.",
                  "Benign corpus is hand-written, small, and not representative of live traffic.",
                  "Sources contain malformed, inert or context-specific strings; denominator inclu
des every sampled line."],
```

```
        "generated_at_utc": datetime.now(timezone.utc).isoformat(timespec="seconds"),
    }


def to_html(result: dict) -> str:
    rows = "".join("<tr><th>" + escape(name) + "</th><td>" + str(v["detected"]) + "/" + str(v["total"]) + "</td><td>" + f"{v['rate']*100:.1f}%" + "</td></tr>" for name, v in result["families"].items())
    hits = "".join(f"<tr><td>{escape(key)}</td><td>{value}</td></tr>" for key, value in result["per_rule_hits"].items())
    notes = "".join(f"<li>{escape(note)}</li>" for note in result["notes"])
    return ("<!doctype html><html lang='en'><head><meta charset='utf-8'><title>WAF local benchmark</title>"
            "<style>body{font:16px system-ui;margin:2em auto;max-width:820px;color:#18263a;background:#f7f9fc}"
            "main{background:white;padding:2em;border-radius:12px;box-shadow:0 2px 15px #ddd}"
            "table{border-collapse:collapse;width:100%}td,th{padding:.6em;border-bottom:1px solid #dae0e8;text-align:left}</style>"
            "</head><body><main><h1>WAF local benchmark</h1>"
            f"<p>Mode: {escape(result['mode'])} · threshold: {result['threshold']} · {escape(result['split'])} split</p>"
            f"<p><strong>{result['malicious']['detected']}/{result['malicious']['total']}</strong> malicious strings detected"
            f" ({result['malicious']['detection_rate']*100:.1f}%). "
            f"<strong>{result['benign']['false_positives']}/{result['benign']['total']}</strong> benign strings flagged"
            f" ({result['benign']['false_positive_rate']*100:.1f}%).</p>"
            f"<h2>Detection by source family</h2><table><tr><th>Family</th><th>Detected / total</th><th>Rate</th></tr>{rows}</table>"
            f"<h2>Rule hits</h2><table><tr><th>Rule</th><th>Hits</th></tr>{hits}</table>"
            f"<h2>Method &amp;amp; limitations</h2><ul>{notes}</ul></main></body></html>")


def save(result: dict, destination: Path) -> None:
    destination.mkdir(parents=True, exist_ok=True)
    (destination / "benchmark.json").write_text(json.dumps(result, indent=2) + "\n", encoding="utf-8")
    (destination / "benchmark.html").write_text(to_html(result), encoding="utf-8")

```

```python
"""Bounded, heuristic anomaly scoring. Not an SQL/HTML parser or production WAF.

The explicit decode budget is intentionally small; each original input is scored
once per normalized representation, and each rule counts at most once per field.
"""
from __future__ import annotations
from dataclasses import asdict, dataclass
from collections import Counter
import re
import unicodedata
from urllib.parse import unquote

MAX_FIELD = 4096
MAX_TOTAL = 16384
MAX_FIELDS = 64
MAX_DECODE_ROUNDS = 2

@dataclass(frozen=True)
class Rule:
    id: str
    family: str
    severity: str
    score: int
    pattern: re.Pattern[str]


def rule(id: str, family: str, severity: str, score: int, pattern: str) -> Rule:
    return Rule(id, family, severity, score, re.compile(pattern, re.IGNORECASE | re.ASCII))

# Bounded input means regex scans are bounded; no nested repetitions in patterns.
RULES = (
    rule("SQL01", "sqli", "critical", 8, r"(?:\bunion\s+(?:all\s+)?select\b|\bselect\s+(?:\*|[\w.]+(?:\s*,\s*[\w.]+)+)\s+from\b|\bdrop\s+table\b|\b(?:or|and)\s+\d+\s*=\s*\d+\b)"),
    rule("SQL02", "sqli", "high", 6, r"(?:\b(?:or|and)\s+['\"\w]+\s*=\s*['\"\w]+\s*(?:--|#|/\*)|\b(?:sleep|benchmark|pg_sleep|waitfor\s+delay)\s*\(|\b(?:information_schema|xp_cmdshell|load_file)\b)"),
    rule("SQL03", "sqli", "medium", 4, r"(?:['\"]\s*(?:or|and)\b|\b(?:isnull|exists|extractvalue|updatexml)\s*\(|\b(?:having|group\s+by)\s+\d+\b|--\s*$|/\*.*?\*/)"),
    rule("SQL04", "sqli", "high", 6, r"(?:\b(?:or|and)\s+[\w\x27\x22]+\s*=\s*[\w\x27\x22]+\s*(?:--|#|$)|\bwaitfor\s+delay\b|\bdeclare\s+@\w+|\bexec\s+sp_\w+)"),
    rule("XSS01", "xss", "critical", 8, r"(?:<\s*/?\s*(?:script|iframe|svg|object|embed|img|math)\b|<\s*\w+\s+[^>]{0,120}\bon\w+\s*=)"),
    rule("XSS02", "xss", "high", 7, r"(?:\bon(?:error|load|click|mouseover|focus|begin)\s*=|\bjavascript\s*:|\bdata\s*:\s*text/html|&#x?[0-9a-f]{2,7};|\bdocument\.(?:cookie|write|location)\b)"),
    rule("XSS03", "xss", "medium", 4, r"(?:\b(?:alert|prompt|confirm)\s*\(|<\s*/?\s*\w+[^>]{0,80}>)"),
    rule("PATH01", "traversal", "critical", 8, r"(?:\.\.[/\\]|[/\\]\.\.[/\\]|\.\.%2[fF]|%2[eE]%2[eE][/\\])"),
    rule("PATH02", "traversal", "high", 7, r"(?:[/\\]etc[/\\](?:passwd|shadow|hosts)\b|\bwin(?:dows|nt)[/\\]system32\b|\bboot\.ini\b|\bproc[/\\]self[/\\]environ\b)"),
    rule("PATH03", "traversal", "high", 7, r"(?:[/\\](?:etc|var[/\\]log|proc[/\\]self)[/\\]|[/\\]\.ssh[/\\]|\.\.2f\.\.2f|\bphp\.ini\b|\bboot\.ini\b)"),
    rule("CMD01", "cmdi", "critical", 8, r"(?:\$\([^)]{1,160}\)|`[^`]{1,160}`|\b(?:sh|bash|cmd(?:\.exe)?|powershell)\s+(?:-c|/c|-[a-z]*command)\b)"),
    rule("CMD02", "cmdi", "high", 6, r"(?:[;|&]\s*(?:cat|id|whoami|curl|wget|ping|sleep|nslookup|echo|sh|bash|rm|ls)\b|\|\||&&\s*(?:cat|id|whoami|echo|sleep|sh|bash)\b|\$\{(?:IFS|PATH|HOME)\})"),
)


def canonical_forms(raw: str) -> tuple[str, ...]:
    """NFKC + at most two URL decodings; deduplicate forms. HTML entities are not decoded."""
    if len(raw) > MAX_FIELD:
        raise ValueError("field size limit exceeded")
    forms: list[str] = []
    current = unicodedata.normalize("NFKC", raw)
```

```
    for _ in range(MAX_DECODE_ROUNDS + 1):
        if current not in forms:
            forms.append(current)
        decoded = unicodedata.normalize("NFKC", unquote(current, errors="replace"))
        if decoded == current or len(decoded) > MAX_FIELD:
            break
        current = decoded
    return tuple(forms)


@dataclass
class Finding:
    rule_id: str
    family: str
    severity: str
    score: int
    field: str  # path/query/body/header, never a user-provided key or value

    def as_dict(self) -> dict:
        return asdict(self)


@dataclass
class Verdict:
    score: int
    blocked: bool
    findings: list[Finding]
    reason: str = ""

    def as_dict(self) -> dict:
        return {"score": self.score, "blocked": self.blocked,
                "reason": self.reason, "findings": [f.as_dict() for f in self.findings]}


class Engine:
    def __init__(self, threshold: int = 6, mode: str = "block"):
        if not 1 <= threshold <= 100:
            raise ValueError("threshold must be 1..100")
        if mode not in {"monitor", "block"}:
            raise ValueError("mode must be monitor or block")
        self.threshold, self.mode = threshold, mode
        self.rule_hits: Counter[str] = Counter()
        self.requests = 0
        self.blocked_requests = 0

    def inspect(self, fields: list[tuple[str, str]]) -> Verdict:
        """Inspect already-separated fields; do not pass attacker-controlled names as types."""
        self.requests += 1
        if len(fields) > MAX_FIELDS or sum(len(v.encode("utf-8", errors="replace")) for _, v in fi
elds) > MAX_TOTAL:
            return self._limit("request size/field count limit exceeded")
        findings: list[Finding] = []
        for field, value in fields:
            if field not in {"path", "query", "body", "header"}:
                raise ValueError("unknown field type")
            if len(value) > MAX_FIELD or len(value.encode("utf-8", errors="replace")) > MAX_FIELD:
                return self._limit("field size limit exceeded")
            seen: set[str] = set()
            for form in canonical_forms(value):
                for r in RULES:
                    if r.id not in seen and r.pattern.search(form):
                        seen.add(r.id)
                        findings.append(Finding(r.id, r.family, r.severity, r.score, field))
                        self.rule_hits[r.id] += 1
        score = sum(f.score for f in findings)
```

08-waf-middleware/src/engine.py (continued)

```
        blocked = self.mode == "block" and score >= self.threshold
        self.blocked_requests += blocked
        return Verdict(score, blocked, findings)

    def _limit(self, reason: str) -> Verdict:
        blocked = self.mode == "block"
        self.blocked_requests += blocked
        return Verdict(self.threshold, blocked, [], reason)

    def stats(self) -> dict:
        return {"requests": self.requests, "blocked": self.blocked_requests,
                "per_rule_hits": dict(sorted(self.rule_hits.items()))}

```

```python
"""Flask before-request WAF and JSONL audit logger with no raw request data."""
from __future__ import annotations
from collections.abc import Callable
import json
import logging
from pathlib import Path
from flask import Flask, g, jsonify, request
from .engine import Engine, MAX_FIELDS, MAX_FIELD, MAX_TOTAL, Verdict

# Skip transport/hop-by-hop headers; secrets are NEVER written to the audit log.
# A header's value can be scanned, but only for this compact allowlist.
SENSITIVE_HEADERS = {"authorization", "cookie", "proxy-authorization", "set-cookie"}


def extract_fields() -> tuple[list[tuple[str, str]], str | None]:
    """Read raw bytes once within bounds; reject unsupported/oversized types explicitly.

    Flask query parsing decodes values once; Engine canonicalizes at most two more
    times. That upper bound is disclosed rather than pretending to parse every proxy.
    """
    fields: list[tuple[str, str]] = []
    if len(request.query_string) > MAX_TOTAL:
        return fields, "query size limit exceeded"
    if len(request.environ.get("RAW_URI", request.path).encode("utf-8", errors="replace")) > MAX_T
OTAL:
        return fields, "path size limit exceeded"
    # RAW_URI is useful if fronted by a server that preserves it; PATH_INFO fallback.
    raw_uri = request.environ.get("RAW_URI", request.path)
    path = raw_uri.split("?", 1)[0]
    fields.append(("path", path))
    for key, val in request.args.items(multi=True):
        fields.extend([("query", key), ("query", val)])
    for name, val in request.headers.items():
        if name.lower() not in SENSITIVE_HEADERS:
            fields.append(("header", val))
            # Avoid parsing unbounded header collections.
            if len(fields) > MAX_FIELDS:
                return fields, "header/field count limit exceeded"
    if request.content_length is not None and request.content_length > MAX_TOTAL:
        return fields, "body size limit exceeded"
    data = request.get_data(cache=True, as_text=False)
    if len(data) > MAX_TOTAL:
        return fields, "body size limit exceeded"
    if data:
        mimetype = request.mimetype or ""
        if mimetype == "application/json":
            try:
                parsed = json.loads(data)
            except (UnicodeError, ValueError, TypeError, RecursionError):
                return fields, "malformed JSON"
            def walk(node: object, depth: int = 0) -> None:
                if depth > 6 or len(fields) > MAX_FIELDS:
                    raise ValueError("JSON nesting/field limit exceeded")
                if isinstance(node, dict):
                    for k, v in node.items():
                        fields.append(("body", str(k)))
                        walk(v, depth + 1)
                elif isinstance(node, list):
                    for v in node:
                        walk(v, depth + 1)
                elif node is not None:
                    fields.append(("body", str(node)))
            try:
                walk(parsed)
            except ValueError as exc:
```

```
                return fields, str(exc)
        elif mimetype == "application/x-www-form-urlencoded":
            for key, value in request.form.items(multi=True):
                fields.extend([("body", key), ("body", value)])
        elif mimetype.startswith("text/"):
            fields.append(("body", data.decode("utf-8", errors="replace")))
        else:
            return fields, "unsupported body content type (not inspected)"
    if len(fields) > MAX_FIELDS:
        return fields, "field count limit exceeded"
    return fields, None


def audit_event(verdict: Verdict, mode: str) -> dict:
    """No user-controlled path, query, header, key, body, IP, or sample appears here."""
    return {"event": "waf_verdict", "mode": mode, "score": verdict.score,
            "blocked": verdict.blocked, "reason": verdict.reason,
            "rule_ids": [f.rule_id for f in verdict.findings],
            "field_types": sorted({f.field for f in verdict.findings})}


def install_waf(app: Flask, engine: Engine, audit_path: Path | None = None,
                log_event: Callable[[dict], None] | None = None) -> None:
    """Install one before_request hook. Choose the audit sink explicitly."""
    if log_event is None:
        def log_event(event: dict) -> None:
            if audit_path is not None:
                audit_path.parent.mkdir(parents=True, exist_ok=True)
                with audit_path.open("a", encoding="utf-8") as fh:
                    fh.write(json.dumps(event, sort_keys=True) + "\n")
            else:
                logging.getLogger("waf.audit").info(json.dumps(event, sort_keys=True))

    @app.before_request
    def check_request():
        fields, error = extract_fields()
        if error:
            engine.requests += 1
            verdict = engine._limit(error)
        else:
            verdict = engine.inspect(fields)
        g.waf_verdict = verdict
        log_event(audit_event(verdict, engine.mode))
        if verdict.blocked:
            return jsonify({"error": "request rejected", "score": verdict.score}), 403


def create_app(mode: str = "block", threshold: int = 6, audit_path: Path | None = None) -> Flask:
    app = Flask(__name__)
    app.config["MAX_CONTENT_LENGTH"] = MAX_TOTAL
    engine = Engine(threshold=threshold, mode=mode)
    app.extensions["waf_engine"] = engine
    install_waf(app, engine, audit_path=audit_path)

    @app.route("/", methods=["GET", "POST"])
    @app.route("/api/echo", methods=["GET", "POST"])
    @app.route("/api/<path:rest>", methods=["GET", "POST"])
    def echo(rest: str = ""):
        # Deliberately never echo user input; local test target, not a vulnerable app.
        return jsonify({"ok": True, "waf_score": g.waf_verdict.score})

    return app

```

08-waf-middleware/tests/test_benchmark.py

```python
import json
from pathlib import Path
from src.benchmark import evaluate, read_lines, save, to_html, FAMILIES, ROOT


def test_payloads_are_sourced():
    for family in FAMILIES:
        for split in ("dev", "holdout"):
            path = ROOT / "samples" / "payloads" / f"{family}_{split}.txt"
            assert "# SOURCE: https://raw.githubusercontent.com/" in path.read_text()
            assert len(read_lines(path)) > 20


def test_benchmark_denominators_and_reports(tmp_path):
    r = evaluate("dev")
    total = sum(v["total"] for v in r["families"].values())
    assert total == r["malicious"]["total"]
    assert r["benign"]["total"] == len(read_lines(ROOT / "samples" / "benign.txt"))
    save(r, tmp_path)
    assert json.loads((tmp_path / "benchmark.json").read_text())["malicious"] == r["malicious"]
    assert "<html" in (tmp_path / "benchmark.html").read_text()
    assert "<script>alert(1)</script>" not in (tmp_path / "benchmark.html").read_text()


def test_monitor_benchmark_rates_same():
    block, monitor = evaluate("dev", mode="block"), evaluate("dev", mode="monitor")
    assert monitor["malicious"] == block["malicious"]
    assert monitor["benign"] == block["benign"]


def test_html_escapes_attacker_supplied_text():
    r = evaluate("dev")
    r["notes"].append("<script>alert(1)</script>")
    assert "&lt;script&gt;" in to_html(r)
    assert "<script>alert(1)</script>" not in to_html(r)

```

08-waf-middleware/tests/test_engine.py

```python
from urllib.parse import quote
import pytest
from src.engine import Engine, canonical_forms, MAX_FIELD, MAX_FIELDS

@pytest.mark.parametrize('payload,family', [
    ("' OR 1=1--", "sqli"),
    ("UNION SELECT password FROM users", "sqli"),
    ("<script>alert(1)</script>", "xss"),
    ("<img src=x onerror=alert(1)>", "xss"),
    ("../../etc/passwd", "traversal"),
    ("..\\..\\windows\\system32", "traversal"),
    ("; cat /etc/passwd", "cmdi"),
    ("$(whoami)", "cmdi"),
])
def test_attack_families(payload, family):
    v = Engine().inspect([("body", payload)])
    assert v.blocked
    assert any(f.family == family for f in v.findings)

@pytest.mark.parametrize('safe', [
    "children's books", "coffee and cake", "select a plan", "O'Reilly's book",
    "engineering/2026/october", "Bash scripting tutorial", "Cat & Dog",
])
def test_safe_input(safe):
    v = Engine().inspect([("query", safe)])
    assert not v.blocked


def test_single_and_double_encoded():
    for encoded in (quote("<script>alert(1)</script>"), quote(quote("<script>alert(1)</script>")))
:
        assert Engine().inspect([("body", encoded)]).blocked


def test_nfkc():
    assert Engine().inspect([("body", "＜script＞alert(1)＜/script＞")]).blocked


def test_three_layers_not_unbounded():
    target = "<script>alert(1)</script>"
    encoded = quote(quote(quote(target)))
    assert len(canonical_forms(encoded)) <= 3
    # Canonicalisation never runs endlessly or silently peels layer 3.
    assert target not in canonical_forms(encoded)


def test_monitor_scores_but_doesnt_block():
    verdict = Engine(mode="monitor").inspect([("body", "<svg onload=alert(1)>")])
    assert verdict.score >= 6
    assert not verdict.blocked


def test_configurable_threshold():
    payload = "<script>alert(1)</script>"
    assert Engine(threshold=6).inspect([("body", payload)]).blocked
    assert not Engine(threshold=30).inspect([("body", payload)]).blocked


def test_per_rule_hits_count_once_per_field_not_form():
    e = Engine()
    e.inspect([("body", quote("<script>alert(1)</script>"))])
    assert e.rule_hits["XSS01"] == 1
    e.inspect([("body", "<script>alert(1)</script>")])
    assert e.rule_hits["XSS01"] == 2

```

```
65
66 def test_oversized_fields_and_count_fail_closed():
67     e = Engine()
68     assert e.inspect([("body", "a" * (MAX_FIELD + 1))]).blocked
69     assert e.inspect([("body", "x")] * (MAX_FIELDS + 1)).blocked
70     assert not Engine(mode="monitor").inspect([("body", "a" * (MAX_FIELD + 1))]).blocked
71
72
73 def test_invalid_config():
74     with pytest.raises(ValueError): Engine(threshold=0)
75     with pytest.raises(ValueError): Engine(mode="invalid")
76     with pytest.raises(ValueError): Engine().inspect([("external", "hi")])
77
78
79 def test_stats():
80     e = Engine()
81     e.inspect([("query", "UNION SELECT foo FROM bar")])
82     e.inspect([("query", "hello")])
83     assert e.stats()["requests"] == 2
84     assert e.stats()["blocked"] == 1
85     assert e.stats()["per_rule_hits"]["SQL01"] == 1
86
```

08-waf-middleware/tests/test_middleware.py

```
import json
from pathlib import Path
import pytest
from flask import Flask
from src.engine import Engine, MAX_TOTAL
from src.middleware import create_app, install_waf, audit_event

@pytest.fixture
def client():
    return create_app().test_client()


def test_get_safe(client):
    assert client.get("/api/echo?q=hello+world").status_code == 200


def test_post_json_attack_block(client):
    response = client.post("/api/echo", json={"input": "' OR 1=1--"})
    assert response.status_code == 403
    assert response.get_json()["error"] == "request rejected"


def test_url_query_single_decoding(client):
    assert client.get("/api/echo?q=%3Cscript%3Ealert%281%29%3C%2Fscript%3E").status_code == 403


def test_body_form_and_key(client):
    assert client.post("/api/echo", data={"search": "<svg onload=alert(1)>"}).status_code == 403
    assert client.post("/api/echo", json={"../../etc/passwd": "safe"}).status_code == 403


def test_header_inspection(client):
    assert client.get("/api/echo", headers={"User-Agent": "<script>alert(1)</script>"}).status_cod
e == 403
    assert client.get("/api/echo", headers={"X-Custom-Input": "; cat /etc/passwd"}).status_code ==
 403
    assert client.get("/api/echo", headers={"X-Any-Custom-Header": "<svg onload=alert(1)>"}).statu
s_code == 403


def test_path_inspection(client):
    # Send an otherwise legitimate routed path to catch traversal in RAW_URI.
    assert client.get("/api/../../etc/passwd").status_code == 403


def test_monitor_mode_passes_all():
    client = create_app(mode="monitor").test_client()
    r = client.post("/api/echo", json={"query": "<script>alert(1)</script>"})
    assert r.status_code == 200
    assert r.get_json()["waf_score"] >= 6


def test_large_body_rejected_before_read():
    client = create_app().test_client()
    assert client.post("/api/echo", data="x" * (MAX_TOTAL + 1), content_type="text/plain").status_
code == 403


def test_malformed_json_rejected():
    client = create_app().test_client()
    assert client.post("/api/echo", data=b'{"oops":', content_type="application/json").status_code
 == 403


def test_unsupported_body_type_explicit():
```

```
    client = create_app().test_client()
    assert client.post("/api/echo", data=b"\x00\x01", content_type="application/octet-stream").status_code == 403


def test_repeated_json_fields_limited():
    client = create_app().test_client()
    r = client.post("/api/echo", json={str(n): n for n in range(70)})
    assert r.status_code == 403


def test_redacted_audit_no_raw_sensitive_value(tmp_path):
    path = tmp_path / "audit.jsonl"
    secret = "FAKE_PASSWORD_SHOULD_NEVER_BE_LOGGED"
    app = create_app(audit_path=path)
    client = app.test_client()
    client.post("/api/echo?q=PRIVATE_QUERY", json={"password": secret, "x": "<script>alert(1)</script>"},
                headers={"User-Agent": "PRIVATE_UA", "Authorization": "Bearer FAKE_PRIVATE_TOKEN"})
    raw = path.read_text()
    for forbidden in [secret, "PRIVATE_QUERY", "PRIVATE_UA", "FAKE_PRIVATE_TOKEN", "script", "password", "/api/echo"]:
        assert forbidden not in raw
    data = json.loads(raw)
    assert data["blocked"]
    assert "XSS01" in data["rule_ids"]
    assert data["field_types"] == ["body"]


def test_audit_in_monitor_mode(tmp_path):
    path = tmp_path / "audit.jsonl"
    client = create_app(mode="monitor", audit_path=path).test_client()
    r = client.post("/api/echo", json={"input": "<script>alert(1)</script>"})
    assert r.status_code == 200
    assert json.loads(path.read_text())["blocked"] is False


def test_plug_into_existing_flask_app():
    app = Flask(__name__)
    @app.get("/hello")
    def hello(): return "hello"
    install_waf(app, Engine(mode="block", threshold=6), log_event=lambda event: None)
    assert app.test_client().get("/hello").status_code == 200
    assert app.test_client().get("/hello?q=UNION+SELECT+foo+FROM+bar").status_code == 403

```

08-waf-middleware/.gitignore

```
__pycache__/
.pytest_cache/
*.pyc
*.db
*.log
.venv/
venv/

```

08-waf-middleware/pytest.ini

```
[pytest]
pythonpath = .
testpaths = tests
markers =
    network: requires network access

```

```python
"""Fetch a curated subset of public attack-payload lists for LOCAL benchmarking only.

Source repository: SecLists by Daniel Miessler et al. (MIT license)
    https://github.com/danielmiessler/SecLists

Each family file in samples/payloads/ starts with "# SOURCE: <url>" lines so every
string can be traced back to its public origin. A seeded random subset is kept per
family (so the repo stays small), then split 50/50 into a "dev" half (used while
writing the rules) and a "holdout" half (only scored, never inspected while tuning).

These strings are only ever sent to this project's own Flask test client.
Usage:  python scripts/fetch_payloads.py [--per-family 200] [--seed 7]
"""
from __future__ import annotations

import argparse
import random
from pathlib import Path

import requests

BASE = "https://raw.githubusercontent.com/danielmiessler/SecLists/master/"
SOURCES: dict[str, list[str]] = {
    "sqli": [
        "Fuzzing/Databases/SQLi/Generic-SQLi.txt",
        "Fuzzing/Databases/SQLi/quick-SQLi.txt",
        "Fuzzing/Databases/SQLi/Generic-BlindSQLi.fuzzdb.txt",
    ],
    "xss": [
        "Fuzzing/XSS/human-friendly/XSS-Jhaddix.txt",
        "Fuzzing/XSS/human-friendly/XSS-RSNAKE.txt",
    ],
    "traversal": [
        "Fuzzing/LFI/LFI-Jhaddix.txt",
    ],
    "cmdi": [
        "Fuzzing/command-injection-commix.txt",
        "Fuzzing/UnixAttacks.fuzzdb.txt",
    ],
}
OUT = Path(__file__).resolve().parent.parent / "samples" / "payloads"


def fetch(url: str) -> list[str]:
    resp = requests.get(url, timeout=20, headers={"User-Agent": "waf-benchmark-fetcher/1.0"})
    resp.raise_for_status()
    lines = []
    for raw in resp.text.splitlines():
        line = raw.strip()
        if not line or line.startswith("#") or len(line) > 400:
            continue
        lines.append(line)
    return lines


def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    ap.add_argument("--per-family", type=int, default=200)
    ap.add_argument("--seed", type=int, default=7)
    args = ap.parse_args()
    OUT.mkdir(parents=True, exist_ok=True)
    rng = random.Random(args.seed)
    for family, paths in SOURCES.items():
        urls = [BASE + p for p in paths]
        pool: list[str] = []
```

```
        for url in urls:
            try:
                pool.extend(fetch(url))
            except requests.RequestException as exc:
                print(f"[warn] {url}: {exc}")
        pool = sorted(set(pool))
        rng.shuffle(pool)
        subset = pool[: args.per_family]
        half = len(subset) // 2
        for split, items in (("dev", subset[:half]), ("holdout", subset[half:])):
            path = OUT / f"{family}_{split}.txt"
            header = [f"# SOURCE: {u}" for u in urls]
            header.append(f"# curated: seed={args.seed}, {len(subset)} of {len(pool)} unique lines
, split={split}")
            path.write_text("\n".join(header + items) + "\n", encoding="utf-8")
            print(f"{family:10s} {split:8s} {len(items):4d} payloads -> {path.name}")


if __name__ == "__main__":
    main()

```

08-waf-middleware/scripts/render_chart.py

```python
"""Render chart from benchmark.json; no network needed."""
from pathlib import Path
import json
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
ROOT = Path(__file__).resolve().parent.parent
report = json.loads((ROOT / "outputs" / "benchmark.json").read_text())
names = list(report["families"])
rates = [100 * report["families"][n]["rate"] for n in names]
fig, ax = plt.subplots(figsize=(8.2, 4.5))
fig.patch.set_facecolor("#f7f9fc")
ax.set_facecolor("#ffffff")
bars = ax.bar(names, rates, color=["#305cbe", "#7c3aed", "#0891b2", "#138a63"])
for bar, n in zip(bars, names):
    v = report["families"][n]
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height()+2,
            f"{v['detected']}/{v['total']}", ha="center", fontsize=10)
ax.set_ylim(0, 110)
ax.set_ylabel("Detection rate (%)")
ax.set_title("Local WAF benchmark · curated public payloads, held-out split", fontsize=12)
ax.spines[["top", "right"]].set_visible(False)
ax.grid(axis="y", alpha=.15)
fig.tight_layout()
out = ROOT / "outputs" / "detection_by_family.png"
fig.savefig(out, dpi=135, bbox_inches="tight")
print(out)

```